

0.59. Tourner en s'éloignant : spirale

`spirale(x, y, tuile, taille, sens, vitesse)` : la lettre part du centre et tourne en s'éloignant, jusqu'au bord d'un carré de cette taille.

1. Ce qu'il faut comprendre

Une spirale tourne en s'éloignant du centre. Chaque branche est un peu plus longue que la précédente.

Ce qui est nouveau ici : `spirale(x, y, tuile, taille, sens, vitesse)`, **six** réglages : ceux de `carre`, sans les `tours` (une spirale ne se répète pas : elle s'élargit jusqu'au bout).

La règle des branches : 1 pas, 1 pas, 2 pas, 2 pas, 3 pas, 3 pas... La longueur **augmente d'un pas toutes les deux branches**. La spirale s'arrête quand elle a atteint le bord du carré de `taille`, soit des branches de $2 \times$ taille pas, et une dernière branche ferme le carré.

Le sens : `1` tourne comme les aiguilles d'une montre (droite, bas, gauche, haut, et on recommence) ; `-1` dans l'autre sens (bas, droite, haut, gauche).

À la fin, la lettre revient au centre en diagonale.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <spirale>` → [0.58.3. La fonction spirale\(\) — tourner en spirale](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1347 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
spirale()      442 o ← main() — une fonction fournie par le langage
main()         34 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() — efface des cases
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 3      39 o sur place, dans spirale() + AdresseCase (10 o)
effacer() × 2    30 o sur place, dans spirale() + AdresseCase (10 o), EffacerCases (12 o)
image() × 3      9 o sur place, dans spirale(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :
spirale() × 1 27 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <spirale> // une tuile qui tourne en spirale
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Une spirale autour de la case (10, 8) :
    //
    // spirale(10, 8, ALPHABET[0], 3, 1, 250);
    //      | | |           | | |
    //      | | |           | | +-- vitesse : 250 ms par pas
    //      | | |           | +----- sens : 1, droite, bas, gauche, haut...
    //      | | |           +----- taille : 3, jusqu'au bord d'un carré 7 × 7
    //      | | +----- tuile : la lettre A
    //      +---+----- x, y : le centre (10, 8)
    //
    // Les branches, avec taille 3 (le plus long : 2 × 3 = 6 pas) :
    //
    // branche : 1 2 3 4 5 6 7 8 9 10 11 12 13
    // longueur : 1 1 2 2 3 3 4 4 5 5 6 6 6
    // vers :    → ↓ ← ↑ → ↓ ← ↑ → ↓ ← ↑ →
    //
    // Le début, chaque case marquée du numéro de sa branche :
    //
    // colonne : 9 10 11 12
    // ligne 7 : 4 5 5 5 A : le centre (10, 8)
    // ligne 8 : 4 A 1 . 1 : un pas à droite
    // ligne 9 : 3 3 2 . 2 : un pas en bas
    //           3 : deux pas à gauche
    //           4 : deux pas en haut
    //           5 : trois pas à droite... et ça grandit
    spirale(10, 8, ALPHABET[0], 3, 1, 250);

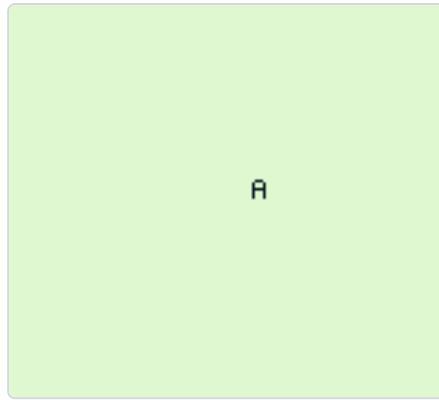
    while (true) {
        image(); // la spirale est finie : le A est revenu au centre
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



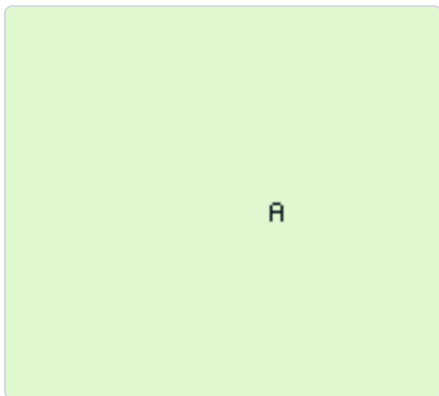
la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui tourne autour de (10, 8) en s'éloignant un peu plus à chaque tour, puis revient au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le premier petit tour : (11, 8), (11, 9), (9, 9), (9, 7)
- ✓ le deuxième, plus grand : (12, 10), (8, 10)
- ✓ jusqu'au bord du 7×7 : (7, 5), (13, 5)
- ✓ pas plus loin que la taille 3
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.59.1. Tourner en s'éloignant : spirale — de base, ailleurs** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++